AI prompts and Responses:

How do I begin by doing these?: - Connect prioritization decisions back to persona needs and journey pain points

from Milestone 1.

- Think about this exercise in terms of what do we need to launch, versus what can

wait for later milestones.

- Create a Project kanban board in GitHub- this is one think I will be grading.

To get started on Milestone 2, you'll need to bridge the gap between your "empathy" work from Milestone 1 and the actual technical build. Here is a step-by-step guide to tackling these three specific tasks.

1. Connecting Prioritization to Personas

The goal here is to justify **why** a feature is a priority based on the human you're building it for.

- **Review your Milestone 1 Personas:** Look at their "Frustrations" or "Pain Points."
- **The "So That" Method:** For every feature you list, write a brief sentence connecting it to a persona.
    - *Example:* "We are prioritizing the **Product Catalog** because our persona, 'Busy Beth,' needs to quickly browse items without navigating multiple pages (Addressing the 'Time Constraint' pain point)."
- **Map to the Journey:** If your user journey showed a "dip" in satisfaction during the checkout phase, your **Shopping Cart** and **Guest Checkout** features should be marked as "High Priority."

2. Defining the "Launch" vs. "Later" (MVP)

In industry terms, you are defining your **Minimum Viable Product (MVP)**. You want to separate "Must-Haves" from "Nice-to-Haves."

Category	Description	Examples

Needs for Launch (MVP)	Critical path features. Without these, the app doesn't function or solve the core problem.	User Login, Product Catalog, Shopping Cart.
Can Wait (V2 / V3)	Features that enhance experience but aren't required for a basic transaction.	Reviews and Ratings, Social Media Sharing, Advanced Analytics.

Pro Tip: Look at the "If you are missing the following features, add them" list in your PDF. Since those are required for the grade, they should almost certainly be in your "Launch" or "In Progress" categories.

3. Creating the GitHub Kanban Board

GitHub Projects is a powerful tool for this. Here is how to set it up:

1. **Navigate to your Repo:** Go to your project repository on GitHub.
2. **Click "Projects":** Found in the top navigation bar of the repo.
3. **Create New Project:** Select the **"Table"** or **"Board"** template (Board is the classic Kanban view).
4. **Set Up Columns:** Standard Kanban columns are:
    a. **Todo:** Features planned but not started.
    b. **In Progress:** What you are currently coding/designing.
    c. **Done:** Completed tasks.
    d. *Optional:* **Backlog** (for the "wait for later" features).
5. **Add Your Features:** Convert the features from your Milestone 1 list into "Issues" or "Draft Tasks" on this board.
6. **Add Labels/Priorities:** Use GitHub labels (e.g., `Priority: High`, `Phase: MVP`) to show the instructor your thought process.

Your Next Step

Would you like me to help you draft the **Architecture Decision Record (ADR)** for the technologies you're considering using for this project?

This is the list of features I am planning to implement for this project, what features do you think are related to the user needs from most important to least?

Based on your Issues list and the persona of **Marcus Chen**, I have ranked your features from most to least critical.

Marcus's primary "pain points" are **security (scams)**, **inefficiency (scouring multiple platforms)**, and **privacy (giving out phone numbers)**.

1. The "Critical Path" (Must-Have for Launch)

These solve Marcus's biggest frustrations immediately.

- **OSU Single Sign-On (SSO):** This is your #1 priority. It eliminates the "bot/scammer" issue Marcus faces on Facebook by ensuring only verified Buckeyes can enter.
- **Product Listing Creation:** Marcus can't sell his monitor without this.
- **Course-Specific Search:** This directly helps Marcus find his "Hibbeler Dynamics" book without digging through unrelated posts.
- **Secure In-App Messaging:** This solves his privacy concern so he doesn't have to share his Discord handle or phone number.

2. The "High Utility" (Build Next)

These make the experience better than the current "messy" alternatives he uses.

- **User Trust Ratings:** Adds a second layer of safety beyond SSO.
- **Safe-Swap Meetup Scheduler:** Helps Marcus coordinate the monitor exchange safely on campus.
- **Item Status Management:** Prevents Marcus from messaging people for books that are already sold (a common pain point in old Discord threads).

3. The "Enhancements" (Later Milestones)

These are great but aren't strictly necessary for Marcus to complete his primary goals of buying/selling.

- **Favorite/Watch List:** Helpful for Marcus to track book prices over time.
- **Book Scanner (ISBN):** Makes it faster for him to list his books.
- **Ticket Marketplace:** A nice secondary feature, but secondary to his immediate need for textbooks and gear.

4. The "Nice-to-Haves" (Backlog)

- **Campus News & Event Feed:** Marcus is a "Busy Engineering Student"—he likely wants to get in, trade, and get out. This feature might be distracting for his specific persona.
- **Accessibility Mode:** Important for general UX, but can be a secondary focus after core trading works.

How to use this for your Milestone 2 grade:

1. Go to your Kanban Board.
2. Move **SSO**, **Listing Creation**, and **Secure Messaging** into your **"In Progress"** or **"Todo"** columns immediately.
3. Label them as `MVP` or `High Priority`.

Would you like me to help you write the specific "justification sentences" for these top 4 features to put in your README?

Can you help me relate all the prioritization decisions to all 21 features in order or most to least important for Marcus Chen?

To get full credit for Milestone 2, you need to show the instructor that every single one of your **21 Issues** has been thought through in the context of your persona, **Marcus Chen**.

Here is the complete prioritization of your features, mapped to Marcus's needs, ranked from most to least important for your **Launch (MVP)**.

Phase 1: The "Must-Haves" (Launch/Critical)

These solve Marcus's primary pain points: Security, Privacy, and Efficiency.

Priority	Issue #	Feature Name	Persona Justification (The "So That")

1	#1	**OSU Single Sign-On (SSO)**	**So that** Marcus feels safe knowing he is trading with real students, not bots or scammers.
2	#2	**Product Listing Creation**	**So that** Marcus can quickly list his monitor for sale with photos and details.
3	#3	**Course-Specific Search**	**So that** Marcus can find his "Hibbeler Dynamics" book instantly without scrolling.
4	#4	**Secure In-App Messaging**	**So that** Marcus can negotiate prices without giving out his personal phone number.
5	#5	**User Trust Ratings**	**So that** Marcus can verify a buyer's reputation before meeting them in person.
6	#9	**Item Status Management**	**So that** Marcus doesn't waste time messaging sellers for items already sold.
7	#20	**Shopping Cart**	**Required for Launch:** Allows Marcus to collect multiple books from different sellers at once.

Phase 2: High Utility (Build Next)

These improve the experience but aren't strictly required for the very first transaction.

Priority	Issue #	Feature Name	Persona Justification
8	#6	**"Safe-Swap" Scheduler**	Helps Marcus coordinate a physical meetup at a campus landmark safely.
9	#19	**Reviews and Ratings**	**Required for Milestone:** Builds long-term community trust for Marcus.
10	#17	**Item Condition Filtering**	Helps Marcus find "Like New" monitors specifically for his setup.

11	#8	**"Wanted" Post Postings**	Marcus can post an ad for a book he can't find yet.
12	#7	**Integrated Digital Payments**	Convenient, but many students still prefer Venmo/Cash outside the app.

Phase 3: Infrastructure & Admin (Milestone Requirements)

*These are ranked lower for Marcus specifically, but are **mandatory** for your grade.*

Priority	Issue #	Feature Name	Reason for Priority
13	#21	**Cloud Deployment**	Graded requirement to ensure the app is live.
14	#18	**Admin Dashboard**	Necessary for you to manage the site and ban bad actors/scammers.

Phase 4: Enhancements (Future Milestones/Backlog)

Nice-to-haves that Marcus would enjoy once the core app works.

Priority	Issue #	Feature Name	Status
15	#13	**Favorite/Watch List**	Helps Marcus track prices of high-end engineering gear.
16	#11	**Book Scanner (ISBN)**	Speed improvement for listing books.
17	#15	**Low-Price Alerts**	Automated efficiency for a "Busy Student."
18	#12	**Ticket Marketplace**	High demand, but requires a separate verification logic.

19	#10	**Guest/Parent View**	Useful for parents buying for students, but not Marcus's core goal.
20	#16	**Accessibility Mode**	Important for inclusivity, but usually follows core feature build.
21	#14	**Campus News Feed**	Lowest priority as it doesn't help Marcus buy/sell items directly.

Next Steps for You:

1. **Kanban Board:** Go to your **Projects** and move Issues #1 through #7 into your "In Progress" or "Todo" column.
2. **README:** Use the "So That" sentences from the first table above to fill out your "Feature Prioritization" section in your README.md.

Would you like me to help you create the Architecture Diagram or the ERD next?

Feature Prioritization & Persona Mapping

The following features have been prioritized based on the needs of our primary persona, **Marcus Chen**. Marcus is a busy engineering student who values security, privacy, and efficiency.

Phase 1: Launch (MVP - Critical Priority)

These features address Marcus's core pain points regarding scams and privacy.

Feature	Issue #	Persona Justification (The "So That")
OSU Single Sign-On (SSO)	#52	**So that** Marcus feels safe knowing he is trading with real students, not bots.

Product Listing Creation	#51	**So that** Marcus can quickly list his monitor for sale with photos and details.
Course-Specific Search	#50	**So that** Marcus can find his "Hibbeler Dynamics" book instantly without scouring.
Secure In-App Messaging	#49	**So that** Marcus can negotiate without giving out his personal phone number.
User Trust Ratings	#48	**So that** Marcus can verify a buyer's reputation before meeting them.
Item Status Management	#44	**So that** Marcus doesn't waste time on items that are already sold.
Shopping Cart	#29	**Required for Launch:** Allows Marcus to batch purchases from different sellers.

Phase 2: High Utility (Build Next)

These improve the experience but aren't strictly required for the first transaction.

Feature	Issue #	Persona Justification
"Safe-Swap" Scheduler	#47	Helps Marcus coordinate a physical meetup at a campus landmark safely.
Reviews and Ratings	#32	**Required for Milestone:** Builds long-term community trust for Marcus.
Item Condition Filtering	#35	Helps Marcus find "Like New" monitors specifically for his setup.
"Wanted" Postings	#45	Allows Marcus to post an ad for a book he can't find yet.
Digital Payments	#46	Convenient, though students may still use Venmo/Cash outside the app.

Phase 3: Infrastructure & Admin (Grading Requirements)

Feature	Issue #	Reason for Priority
Cloud Deployment	#23	Graded requirement to ensure the application is live and accessible.
Admin Dashboard	#34	Necessary for site management and banning potential scammers.

Phase 4: Future Enhancements (Backlog)

Feature	Issue #	Benefit to Marcus
Favorite/Watch List	#40	Helps track prices of high-end engineering gear.
Book Scanner (ISBN)	#42	Speed improvement for listing books.
Low-Price Alerts	#38	Automated efficiency for a "Busy Student."
Ticket Marketplace	#41	High demand, but requires separate verification logic.
Guest/Parent View	#43	Useful for parents, but not Marcus's core goal.
Accessibility Mode	#37	Essential for inclusivity in later versions.
Campus News Feed	#39	General community engagement (Non-core feature).

Can you list the items I need to put in kanban to do in order (21)?

To get full credit for your **Project Kanban board**, you should have all 21 items visible. In a Kanban board, "Order of Importance" is shown by **vertical placement** (top to bottom) within your columns.

Here is the exact order you should follow to match your [README.md]() prioritization for **Marcus Chen**.

1. The "In Progress" Column (Top Priority)

Drag these to your **In Progress** column first. These are the "Must-Haves" for your launch.

1. **#1 OSU Single Sign-On (SSO)** (Most Important for Security)
2. **#2 Product Listing Creation**
3. **#3 Course-Specific Search**
4. **#4 Secure In-App Messaging**
5. **#20 Shopping Cart**

2. The "To Do" Column (High to Low Priority)

Keep these in your **To Do** column, but stack them in this vertical order so the instructor sees your "roadmap."

High Utility (Build Next):

6. **#5 User Trust Ratings**

7. **#9 Item Status Management**

8. **#6 "Safe-Swap" Meetup Scheduler**

9. **#19 Reviews and Ratings** (Milestone Requirement)

10. **#17 Item Condition Filtering**

11. **#8 "Wanted" Post Postings**

12. **#7 Integrated Digital Payments**

Infrastructure & Requirements:

13. **#21 Cloud Deployment** (Milestone Requirement)

14. **#18 Admin Dashboard & Analytics** (Milestone Requirement)

Future Enhancements (Backlog):

15. **#13 Favorite/Watch List**

16. **#11 Book Scanner (ISBN Integration)**

17. **#15 Automated Low-Price Alerts**

18. **#12 Ticket Marketplace**

19. **#10 Guest/Parent View**

20. **#16 Accessibility Mode**

21. **#14 Campus News & Event Feed** (Lowest Priority)

Pro-Tip for the Grade

To add these 21 items quickly:

1. Go to your **Project Board**.
2. In the **To do** column, click **+ Add item**.
3. Type # followed by the number (e.g., #52) and press Enter.
4. Repeat until all 21 are on the board.
5. Drag the top 5 into **In Progress**.

Would you like me to help you draft the Architecture Decision Record (ADR) next?

Can you help me to create: High-level system architecture showing major components (frontend, backend,
database) and how they interact.
- Connect architecture decisions back to user needs from Milestone 1
- This can be any document format you choose. It must be in your GitHub repo
under the docs folder.

Since you are building the **Buckeye-Marketplace**, your architecture needs to be robust enough to handle secure transactions and student data while remaining simple enough for a "busy student" like Marcus Chen to use.

Below is the conceptual layout for your **System Architecture**. You can use a tool like **Lucidchart**, **Draw.io**, or even a clean PowerPoint slide to create the actual image file to upload to your `docs/` folder.

1. High-Level System Architecture Diagram

The diagram should show how the user's browser interacts with your server and how that server talks to the database and external services (like OSU's login).

The Component Flow:

1. **Frontend (The Client):** Built with **React**. This is what Marcus sees on his phone or laptop.
2. **API Gateway / Backend (The Server):** Built with **Node.js/Express**. This acts as the "brain," processing requests from the frontend.
3. **Database:** A relational database like **PostgreSQL** or **MySQL** to store users, items, and messages.
4. **External Services:** * **OSU Shibboleth/OIDC:** For the Single Sign-On (SSO) login.
    a. **Cloud Storage (e.g., AWS S3):** To store the high-quality photos Marcus uploads of his monitor.

2. Connecting Architecture to User Needs

In your documentation, you must explain **why** you chose this setup based on Milestone 1. Here is a draft you can include in your `docs/architecture_notes.md` or directly in your `README.md`:

- **Decoupled Frontend/Backend:** By using **React** separately from the backend, we ensure the app is fast and responsive. Marcus is a "Busy Student"; he won't wait for slow page reloads when searching for a textbook.
- **Relational Database:** We chose a **Relational DB** (PostgreSQL/MySQL) because "Trust" and "Accuracy" are critical for Marcus. We need to ensure that if an item is marked "Sold," it is reflected instantly and accurately across the whole system.
- **SSO Integration (External):** The architecture includes a dedicated path for **OSU Shibboleth**. This directly addresses Marcus's fear of scammers by ensuring every user is a verified Buckeye.

- **Secure API Layer:** All communication between Marcus and other students goes through our Backend API. This keeps his personal contact info private, solving his "Privacy" pain point.

3. How to add this to GitHub

As per the [Milestone 2 instructions](), this must be in your `docs` folder.

1. **Create the Image:** Export your diagram as a `.png` or `.jpg`.
2. **Upload to GitHub:**
    a. Go to your `Buckeye-Marketplace` repo.
    b. Click on your `docs/` folder.
    c. Click **Add file** > **Upload files**.
    d. Upload your diagram (name it `system-architecture.png`).
3. **Link it in your README:** Add this line to your README so the instructor can see it easily:

```
![System Architecture](./docs/system-architecture.png)
```

Your Next Step

I can help you define the **Database Schema (ERD)** now. This will decide exactly what "Tables" you need (like a Users table and an Items table) to support these features. **Would you like to start on the ERD?**

How do you recommend I begin creating the high-level system architecture with components (frontend, backend, database) and how they interact to solve Marcus Chen's problems?

To create a high-level architecture that specifically addresses [Marcus Chen's]() pain points, you should move away from a "generic" box-and-arrow diagram and label it with the specific services that solve his frustrations.

I recommend structuring your diagram and your explanation around these **three core interaction loops** identified in your Milestone 1 Journey Map.

1. The "Trust & Security" Component

Marcus's Pain Point: Anxiety over scammers and "is this available?" bots.

- **The Component: OSU Shibboleth (SSO) Integration.**
- **Interaction:** The Frontend (React) redirects to the OSU login service. The Backend (Node.js/Python) receives a "token" verifying the user is a real student.
- **The Solution:** This eliminates the "fake buyer" issue Marcus faces on Facebook and Reddit.

2. The "Efficiency" Component

Marcus's Pain Point: Wasted time "scouring" multiple platforms for one book.

- **The Component: Elastic Search / Course-Specific API.**
- **Interaction:** The Frontend sends a specific course code (e.g., "MECHENG 2030") to the Backend. The Backend queries the **PostgreSQL/MySQL Database** using an index specifically for "Course Codes."
- **The Solution:** Marcus finds his "Hibbeler Dynamics" book in three clicks instead of thirty minutes of scrolling.

3. The "Privacy & Logistics" Component

Marcus's Pain Point: Giving out his personal phone number or Discord handle to strangers.

- **The Component: Encrypted In-App Messaging.**
- **Interaction:** Users send messages through a WebSocket/API layer. The Backend stores these in the Database without revealing either user's personal email or phone number.
- **The Solution:** Marcus negotiates the monitor sale safely and keeps his personal data private.

Recommended Tools for Creation

Since this needs to live in your `docs` folder on GitHub, use one of these to generate a `.`png or `.svg`:

- **Draw.io (free/web-based):** Great for standard architectural boxes.
- **Lucidchart:** Professional and very clean.

- **Excalidraw:** Excellent for a "whiteboard" style that looks more modern.

How to Document it for the Grade

In your `docs/architecture.md` file, use a table to explain your choices so the instructor can see the connection to Milestone 1 clearly:

Component	Tech Choice	Solves Marcus's Pain Point
Frontend	React (SPA)	**Efficiency:** Provides the "instant" feel Marcus expects as a high-tech user.
Auth	OSU Shibboleth	**Trust:** Stops the "bot" problem he faces in Facebook groups.
Messaging	WebSockets	**Privacy:** Keeps him off Discord/texting for campus trades.
Database	Relational DB	**Accuracy:** Ensures item status ("Sold") is 100% accurate so he doesn't waste time.

Your Next Step

Would you like me to help you define the **Database Schema (ERD)** specifically for Marcus's needs, such as what columns are needed for the "Course-Specific Search"?

Can you create an entry relationship diagram (ERD) from Milestone 1?

I can definitely help you design the **Entity Relationship Diagram (ERD)** based on your Milestone 1 goals and Marcus Chen's needs.

According to your Milestone 2 instructions, you don't need to worry about data types or normalization yet. You just need the **main tables** and **relationships** that support your prioritized features.

Proposed Database Entities for Buckeye-Marketplace

To solve Marcus's problems (scam prevention, efficiency, and privacy), your database needs these four core tables:

Entity (Table)	Purpose for Marcus Chen
Users	Stores verified OSU identity (via SSO) to ensure he's only dealing with students.
Items (Books)	Stores the textbook details, condition, and price so Marcus can find what he needs.
Messages	Enables the "Encrypted In-App Messaging" so he doesn't have to give out his phone number.
Categories	Allows Marcus to filter by "Engineering" or "CSE" to save time.

The Relationships (The "Logic")

Here is how these tables connect (this is what you will draw in **draw.io**):

1. **User to Items (One-to-Many):** One User can list many Items for sale.
2. **User to Messages (One-to-Many):** One User can send/receive many Messages.
3. **Item to Messages (One-to-Many):** One Item (a specific book) can have a conversation thread attached to it.
4. **Category to Items (One-to-Many):** One Category (e.g., "Textbooks") can contain many Items.

How to draw this for your grade:

1. Open draw.io (start a new blank diagram).
2. On the left sidebar, look for the **Entity Relation** shape library.
3. Drag **4 rectangles** onto the canvas and label them: `Users`, `Items`, `Messages`, and `Categories`.
4. Use the "crow's foot" arrows to connect them based on the relationships above.

Why this supports your User Stories:

- **Security:** The `Users` table linked to `Messages` ensures that every chat participant is a verified student.
- **Efficiency:** The `Categories` table allows Marcus to skip the junk and find his specific engineering books instantly.

Would you like me to list the specific fields (like `user_id` or `item_status`) you should put inside those boxes to make it look even more professional?

Next what do you recommend I write: Architecture Decision Records
- What technology are we using and why?
- AI tool usage: Document AI-assisted research

Great job getting those diagrams live! Since the [Milestone 2 instructions]() require an **Architecture Decision Record (ADR)**, this is your chance to explain the "brains" behind the project.

For a student project like Buckeye-Marketplace, I recommend a clean, professional list in your README that justifies your tech stack.

Recommended ADR Content

You can paste this structure directly into your **README.md** and fill in the details:

4. Architecture Decision Records (ADR)

Tech Stack Selection

- **Frontend:** React.js / Next.js
  - *Why:* It allows for a responsive design that Marcus can use on his phone while walking between classes at Hitchcock Hall.
- **Backend:** Node.js with Express
  - *Why:* Fast and scalable for handling real-time messaging between students.
- **Database:** PostgreSQL (or MongoDB)
  - *Why:* Reliable storage for user verification data and item listings.
- **Authentication:** OSU Webauth / OpenID Connect
  - *Why:* To ensure only verified Buckeyes can log in, directly solving Marcus's fear of non-student scammers.

AI Tool Usage Disclosure

- **Tool Used:** Gemini (Google AI)

- **Purpose:** Used for brainstorming the 3-tier system architecture and refining the Entity Relationship Diagram (ERD) to include specific safety features (like the `is_verified` flag) based on Milestone 1 persona research.
- **Human Oversight:** All AI-generated suggestions were manually reviewed and modified to fit the specific requirements of the CSE 4630 Milestone 2 rubric.

Where to put this?

1. Open your **README.md** editor again.
2. Paste this right below your **Database ERD** section.
3. **Commit changes.**

Why this helps your grade:

The rubric on **Page 3** of your Milestone 2 PDF gives 5 points for "Clear, well-justified decisions with AI usage documented." By explicitly mentioning Marcus Chen and the specific AI help, you're hitting the "Excellent" criteria.

Would you like me to help you with the final part of the milestone—the Component Architecture (Atomic Design) list?

How do you recommend I begin adding : Component Architecture
- Using Atomic Design principles, create a component hierarchy..
- Scope the components to the Product Catalog feature for now.?

To wrap up **Section 5** of your Milestone 2, you need to show the instructor how you'll break the UI down into small, reusable pieces.

Since the instructions say to focus on the **Product Catalog** (the page where Marcus Chen would browse for his engineering books), I recommend adding a list that follows the **Atomic Design** hierarchy: Atoms, Molecules, and Organisms.

5. Component Architecture (Atomic Design)

Add this to the bottom of your **README.md**:

```
Markdown
```

```
---
## 5. Component Architecture (Atomic Design)
To ensure a consistent and reusable UI for the Product Catalog, we are following
Atomic Design principles.

### Atoms (Smallest functional units)
* **Button:** Red/Gray action buttons for "View Details" or "Message Seller."
* **Input Field:** Search bar text entry.
* **Badge:** Price tags and condition labels (e.g., "Like New").
* **Typography:** Specific font styles for book titles and headers.

### Molecules (Groups of atoms working together)
* **Search Bar:** Combines the Input atom with a Search Button atom.
* **Item Card Snippet:** Combines a book image, title atom, and price badge.
* **Filter Toggle:** Group of checkboxes for filtering by category (e.g., "CSE,"
"Mechanical").

### Organisms (Complex UI sections)
* **Product Grid:** A collection of Item Card molecules showing all available
textbooks.
* **Navigation Bar:** The top header containing the logo, Search Bar, and User
Profile link.
* **Sidebar Filter:** The section containing all category and price range
Molecules.
```

Why this is the "Finish Line"

By adding this section, you have fulfilled every single requirement in the [Milestone 2 rubric]:

1. **Logical Organization:** You categorized components.
2. **Scoped to Feature:** You focused specifically on the **Product Catalog**.
3. **Persona-Focused:** You mentioned the specific search and filter tools Marcus needs to find his books quickly.

How to submit:

1. **Commit** these last changes to your main `README.md`.
2. Go to **Carmen**.
3. Submit the **Link to your GitHub repo**.
4. Mention in the comments that your diagrams are in the `docs` folder and linked in the `README`.

Would you like me to take one last look at your GitHub repository link to make sure everything is in the right place before you submit it to your instructor?